Projet de Moteur de Recherche
M1 Programmation Spécialisée

Laura Moret

8 novembre 2026

https://github.com/jortanix/projet_prg_spe

Table des matières

1 Spécifications **3**

2 Analyse **3**

2.1 Environnement de travail 3

2.2 Données identifiées 4

2.3 Diagramme de classes 4

3 Conception **4**

3.1 Répartition des tâches 4

3.2 Algorithmes spécifiques 5

3.2.1 Analyse textuelle 5

3.2.2 Score TF-IDF 5

3.2.3 Recherche et similarité 5

3.2.4 Clustering 5

3.2.5 Bonus : Okapi BM25 5

3.2.6 Bonus : analyse comparative de sous-corpus 5

3.3 Problèmes rencontrés et solutions 6

3.4 Exemple d'utilisation 6

4 Validation **7**

4.1 Tests unitaires 7

4.2 Tests globaux 7

5 Maintenance et perspectives **7**

5.1 Évolutions en cours 7

5.2 Perspectives d'amélioration 7

6 Contraintes techniques **8**

7 Conclusion **8**

Résumé

Ce projet porte sur la conception et l'implémentation d'un moteur de recherche capable d'analyser, de structurer et d'interroger un corpus de documents textuels issus de deux sources distinctes : Reddit et arXiv. Il s'appuie sur un modèle orienté objet, un système de scoring TF-IDF calculé manuellement, ainsi qu'une série de fonctionnalités avancées (recherche multi-critères, clustering, recommandations, analyse statistique et export des résultats).

Le travail est organisé en trois parties. La première établit la structure de données (documents, auteurs, corpus avec patron singleton, fabrique de documents) et l'acquisition des données. La deuxième ajoute l'analyse textuelle, le concordancier et le moteur de recherche basé sur la matrice Documents × Termes. La troisième enrichit le moteur avec des fonctionnalités avancées (recherche filtrée, Okapi BM25, clustering, recommandations, statistiques, export) et une interface web, complétées par plusieurs bonus.

1 Spécifications

L'objectif est de construire un moteur de recherche complet permettant de :

- Collecter et structurer des documents textuels depuis des sources externes (Reddit et arXiv) ;
- Analyser le texte : tokenization, suppression des mots vides, construction d'un vocabulaire et calcul de fréquences ;
- Construire une matrice Documents × Termes à partir de scores TF-IDF calculés manuellement ;
- Offrir un moteur de recherche basé sur ces scores, avec filtres et tris ;
- Proposer des fonctionnalités avancées : clustering, recommandations, analyse de fréquence, tendances temporelles, statistiques et export CSV/JSON ;
- Respecter les contraintes techniques imposées (interdiction de scikit-learn et nltk, calculs effectués manuellement).

Le périmètre de la partie 3 prévoit également une interface graphique. Une interface web interactive a été implémentée avec `Streamlit` (`streamlit run app.py`) ; l'interface notebook Jupyter avec `ipywidgets`, prévue par l'énoncé, est présentée dans les perspectives.

2 Analyse

2.1 Environnement de travail

Le projet est développé en Python 3.x. Les bibliothèques utilisées sont limitées aux besoins strictement nécessaires :

- NumPy : calculs numériques et manipulation de vecteurs ;
- SciPy : matrices creuses (`csr_matrix`) et algorithme de clustering K-means ;
- Pandas : organisation des données et export en CSV ;
- Matplotlib et Seaborn : visualisation des résultats (facultatif) ;

— Streamlit : interface web interactive (`app.py`) ;
— Bibliothèque standard : `datetime`, `collections`, `re`, `json`.

2.2 Données identifiées

Deux types de documents sont traités, modélisés sous forme de classes dérivées :

— **Documents Reddit** : titre, auteur, date, URL, texte et nombre de commentaires ;
— **Documents arXiv** : titre, auteur, date, URL, texte et liste de co-auteurs.

Chaque document appartient à un **auteur**, et l'ensemble des documents et des auteurs constitue un **corpus**. Le corpus conserve un vocabulaire global ainsi que la matrice Documents × Termes.

2.3 Diagramme de classes

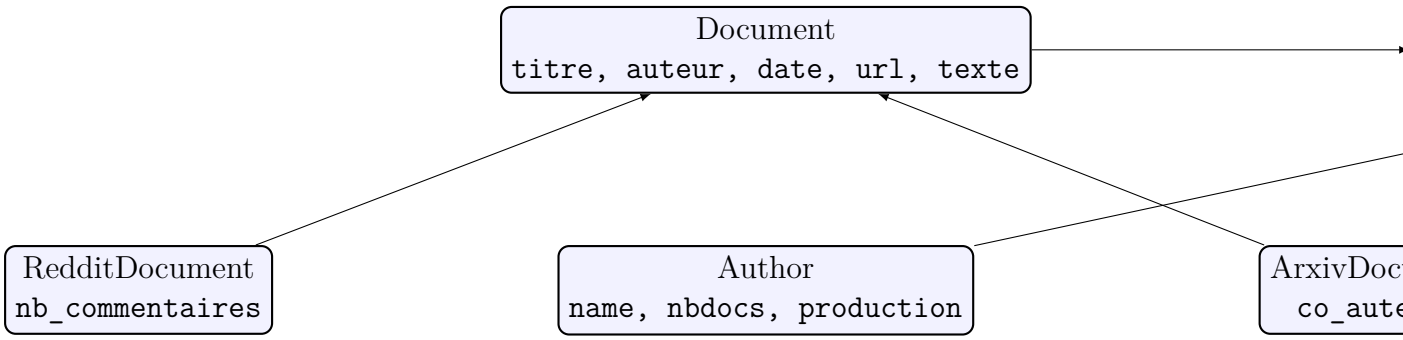


FIGURE 1 – Diagramme de classes simplifié du projet

La classe `Corpus` centralise la recherche, la construction de la matrice et la persistance, tandis que `SearchEngine` encapsule les fonctionnalités avancées (recherche multi-critères, clustering, recommandations, statistiques et export).

3 Conception

3.1 Répartition des tâches

Le projet a été réalisé de manière individuelle. Le travail a été décomposé en trois étapes :

— **Partie 1 – Structure** : création des classes `Document`, `RedditDocument`, `ArxivDocument`, `Author` et `Corpus`, fabrique de documents, acquisition des données et persistance (sauvegarde en TSV) ;
— **Partie 2 – Recherche** : analyse textuelle (tokenization, suppression des mots vides), construction du vocabulaire et de la matrice Documents × Termes, calcul des scores TF-IDF, recherche, similarité et concordancier ;
— **Partie 3 – Avancé** : moteur de recherche à filtres et tris, clustering, recommandations, analyses, statistiques, export CSV/JSON, interface web Streamlit, et fonctions bonus (Okapi BM25, analyse comparative de sous-corpus).

3.2 Algorithmes spécifiques

3.2.1 Analyse textuelle

L'analyse consiste en :
- la tokenization du texte (découpage en termes, gestion de la ponctuation) ;
- la suppression des mots vides et la normalisation en minuscules ;
- la construction d'un vocabulaire global avec les fréquences des termes.

3.2.2 Score TF-IDF

Le poids d'un terme t dans un document d est :

$$TF\text{-}IDF(t,d) = TF(t,d) \times \log\left(\frac{N}{DF(t)}\right)$$

où N est le nombre total de documents et $DF(t)$ le nombre de documents contenant le terme. Les scores sont stockés dans une matrice creuse Documents × Termes pour économiser la mémoire.

3.2.3 Recherche et similarité

La pertinence d'un document pour une requête est évaluée en agrégeant les scores TF-IDF des termes de la requête. La similarité entre deux documents repose sur le cosinus de leurs vecteurs.

3.2.4 Clustering

Le clustering repose sur l'algorithme **K-means** appliqué aux vecteurs de documents issus de la matrice TF-IDF (via SciPy), puis à l'attribution de chaque document au centroïde le plus proche.

3.2.5 Bonus : Okapi BM25

En complément du scoring TF-IDF (cosinus), une mesure **Okapi BM25** est proposée dans `search_bm25`. Le score d'un document d pour une requête est :

$$BM25(d,q) = \sum_{t \in q} IDF(t) \cdot \frac{f(t,d) \cdot (k_1+1)}{f(t,d) + k_1\left(1 - b + b \cdot \frac{|d|}{avgdl}\right)}$$

avec $IDF(t) = \log\left(1 + \frac{N - df(t) + 0.5}{df(t) + 0.5}\right)$. Les paramètres sont fixés aux valeurs classiques $k_1 = 1.5$ et $b = 0.75$. Ce modèle affine le classement en tenant compte de la longueur des documents et de la saturation de la fréquence de terme.

3.2.6 Bonus : analyse comparative de sous-corpus

La méthode `analyser_sous_corpus` compare deux groupes de documents (par exemple `Reddit` et `arXiv`). Chaque groupe est agrégé en un pseudo-document puis un score TF-IDF

local est calculé pour chaque terme. Les termes les plus importants de chaque groupe sont alors comparés : les mots communs mettent en évidence le vocabulaire partagé, tandis que les mots *spécifiques* à chaque groupe révèlent les thématiques propres à chaque source. Appliquée au corpus (10 documents Reddit et 10 arXiv), l'analyse fait ressortir des termes spécifiques à arXiv (par ex. *framework*, *xai*, *belief*, *games*) et à Reddit (par ex. *proof*, *conjecture*, *quantum*), le mot *ai* étant commun aux deux sources.

3.3 Problèmes rencontrés et solutions

— **Gestion des dates** : comparaison et tri par date nécessitaient un format cohérent ; `datetime` a été utilisé et des filtres de période (`date_start`, `date_end`) ont été ajoutés à la recherche avancée.
— **Efficacité de la matrice** : la matrice Documents × Termes pouvant être volumineuse et creuse, elle est représentée en *matrice creuse* (SciPy) afin de limiter l'empreinte mémoire.
— **Héritage et polymorphisme** : les deux types de documents partagent un comportement commun ; l'héritage de `Document` et la redéfinition de `getType()` évitent la duplication de code.
— **Unicité du corpus** : le patron *singleton* garantit qu'une seule instance de corpus existe à travers l'application.
— **Caractères accentués** : la gestion de l'encodage UTF-8 (accents français) a nécessité une configuration adaptée de l'environnement Python et des fichiers de sauvegarde ;
— **Mots vides multilingues** : le corpus étant majoritairement anglophone, la liste de mots vides a dû être complétée par les *stopwords* anglais afin d'éviter que des termes outils (*the*, *and*, *of*...) ne polluent le vocabulaire et l'analyse comparative.

3.4 Exemple d'utilisation

Le fonctionnement du moteur s'illustre par l'enchaînement suivant :

1. Création du corpus puis initialisation automatique de son moteur de recherche ;
2. Recherche avancée : la requête est passée avec éventuellement des filtres (auteur, type, période) et un critère de tri (pertinence, date ou auteur) ;
3. Regroupement des documents similaires en un nombre donné de clusters ;
4. Obtention de recommandations basées sur la similarité avec un document de référence ;
5. Export des résultats de recherche aux formats CSV ou JSON, et consultation de statistiques globales (nombre de documents, termes les plus fréquents, types de documents) ;
6. Interrogation du moteur à travers l'interface web `Streamlit`, qui expose la recherche (TF-IDF ou BM25), les filtres, les statistiques et l'analyse comparative.

4 Validation

4.1 Tests unitaires

Des tests unitaires couvrent chacune des fonctionnalités :

- **Partie 1** : initialisation du corpus et de ses attributs, patron singleton, création des documents Reddit et arXiv, gestion des auteurs, ajout de documents, affichage (`__repr__`) et sauvegarde ;
- **Partie 2** : construction du vocabulaire, dimensions et type de la matrice Documents × Termes, résultats de recherche, calcul de similarité et extraction des termes importants ;
- **Partie 3** : initialisation du moteur de recherche, recherche avancée (filtres, tri), clustering, recommandations, analyses (fréquence, tendances, statistiques) et export en CSV et JSON.

Chaque test renvoie un résultat explicite, et un compteur agrège le nombre de tests réussis et échoués. Des cas d'exception sont également couverts afin de vérifier le comportement en présence de données invalides.

4.2 Tests globaux

Les tests globaux vérifient l'intégration complète du système, c'est-à-dire :

- l'acquisition et la persistance des données (sauvegarde et rechargement du corpus) ;
- la chaîne complète analyse → vocabulaire → matrice TF-IDF ;
- l'enchaînement recherche → filtres → tri → export ;
- la cohérence entre le clustering, les recommandations et les statistiques produites à partir du même corpus.

5 Maintenance et perspectives

5.1 Évolutions en cours

Les fonctionnalités avancées de la partie 3 (recherche filtrée, clustering, recommandations, statistiques, export) ainsi que plusieurs bonus sont implémentés et intégrés au moteur :

- **Interface web** : une interface `Streamlit` (`app.py`) permet d'interroger le corpus sans écrire de code, avec le choix de la mesure de score (TF-IDF ou BM25), un filtre par source et l'affichage des statistiques ;
- **Okapi BM25** : mesure de score alternative au TF-IDF pour affiner le classement des résultats ;
- **Analyse comparative** : comparaison des vocabulaires de deux sous-corpus (Reddit vs arXiv).

5.2 Perspectives d'amélioration

Les axes d'amélioration envisagés sont :

— l'interface notebook Jupyter avec `ipywidgets` et barres de progression, prévue par l'énoncé, en complément de l'interface web ;
— le traitement avancé du langage (lemmatisation, stemming) pour améliorer la qualité du vocabulaire ;
— l'optimisation des performances sur de grands volumes de données (parallélisation, index inversé) ;
— le raffinement du clustering (choix automatique du nombre de clusters, autres métriques de distance) ;
— la prise en charge de formats et de sources de documents supplémentaires.

6 Contraintes techniques

Le projet respecte les contraintes imposées :
— **Interdits** : scikit-learn, nltk et autres bibliothèques d'apprentissage ou de traitement avancé ;
— **Autorisés** : NumPy, SciPy (matrices creuses et K-means), Pandas ;
— **Calcul manuel** : l'analyse textuelle, le vocabulaire, les fréquences et les scores TF-IDF sont implémentés manuellement ;
— **Persistance** : sauvegarde du corpus au format TSV et du vocabulaire dans un fichier séparé.

7 Conclusion

Ce projet a permis de concevoir un moteur de recherche complet, structuré autour de principes orientés objet et reposant sur un calcul manuel du TF-IDF. Il intègre un moteur de recherche à filtres et tris, une interface web, ainsi que des fonctions bonus (Okapi BM25, analyse comparative de sous-corpus), tout en respectant les contraintes techniques fixées. La poursuite du travail porte principalement sur l'interface notebook `ipywidgets` prévue par l'énoncé.

Lien vers le dépôt

Le code source complet est disponible à l'adresse :

https://github.com/jortanix/projet_prg_spe

Références

[1] arXiv API Documentation. https://arxiv.org/help/api/index
[2] Reddit API Documentation. https://www.reddit.com/dev/api/
[3] Scipy Documentation. https://docs.scipy.org/
[4] Pandas Documentation. https://pandas.pydata.org/

[5] TF-IDF Wikipedia. https://fr.wikipedia.org/wiki/TF-IDF